Assignment 3: TypeScript

Complex IT Systems – Theory E2026 (40417) · Section 2, backend development
Units: types, arrays, higher-order functions and closures

Work as a group. Discuss each task before you write anything, then write the code together, and make sure every one of you can explain every line afterwards. What you can do at the end is the point of this; the file you hand in is only the evidence.

Before you start

Download the zip with the starting project from the course website and unpack it. Everything you need is inside. Open that folder in VS Code (File > Open Folder) and open a terminal in it.

Install the dependencies and run the project once, so you know the loop you will be working in:

```
npm install
npm run build
npm start
```

The data is the university database you already know, this time with the students, their enrolments and their grades. The grades are strings on the Danish 7-point scale (12, 10, 7, 4, 02, 00, -3), not numbers: as numbers, 02 and 00 would lose their leading zero. A grade of null means it has not been awarded yet.

Fill in every `TODO` and do not change the signatures. The parameter types and the return types are part of the task too. Uncomment the blocks in `src/index.ts` as you finish each function. The expected result is written beside every line, so you can check your own work without waiting for us.

Part A: Arrays and types

Work in `src/arrays.ts`. Use no `for` loops anywhere in this file: every task here can be done with the array methods.

A1 `gradeReport(studentId)`

Write a function that takes a student's id and returns one block of text with a line for each of that student's enrolments, in the order the enrolments appear in the data. Make each line the course id, the course title, a colon and the grade, and write `not awarded` where the grade is `null`. Return the empty string if the student has no enrolments at all.

```
gradeReport('98988')

BIO-101 Intro. to Biology: 12
BIO-301 Genetics: not awarded
```

An enrolment carries the grade and the course id, but not the title, so for every enrolment you have to find its course.

A2 `averageGrade(studentId)`

Write a function that takes a student's id and returns the average of the grades that student has actually been awarded, rounded to one decimal. Leave the enrolments without a grade out of both the sum and the count. Return 0 when the student has no awarded grades, rather than dividing by zero.

```
averageGrade('00128')   // 11   the average of 12 and 10
averageGrade('98988')   // 12   the ungraded enrolment does not count
averageGrade('70557')   // 0    no enrolments at all
```

The grades are stored as strings, so turn them into numbers before you add them up.

A3 `awaitingGrades()`

Write a function that takes nothing and returns the names of the students who are still waiting for at least one grade, sorted alphabetically, with every name appearing once.

```
awaitingGrades()   // [ 'Tanaka' ]
```

A student with two ungraded enrolments would otherwise turn up twice, so get rid of the repeats before you return the list.

A4 `formatterFor(style)`

Write a function that takes a formatting style and returns another function. The function it returns takes a student and gives back a line of text in that style. Only two styles are allowed, `'short'` and `'long'`, so declare a type that allows just those two values. The compiler will then reject anything else.

```
formatterFor('short')(students[0])   // Zhang
formatterFor('long')(students[0])    // Zhang (Comp. Sci.), 102 credits
```

Try calling `formatterFor('medium')` and read the error you get. Notice when you get it: before the program has run at all.

A5 `raiseCredits(list, deptName, extra)`

Write a function that takes a list of students, the name of a department and a number of credits, and returns a new list in which the students of that department have that many credits more. Leave every other student exactly as they are, and leave the list you were given untouched: build new objects instead of changing the ones you were handed.

```
const raised = raiseCredits(students, 'Physics', 10);

raised[4].totCred     // 66    Peltier, was 56
students[4].totCred   // 56    unchanged
raised[0].totCred     // 102   Zhang, not a physicist
```

The last lines of the Part A block in src/index.ts check that the students array you were given still holds its old values afterwards. Copying the array is not enough on its own: if the copy holds the

same student objects, then raising the credits on one of them changes that student in the original array too.

Part B: Higher-order functions and closures

Work in `src/functions.ts`. Every function here either takes a function, returns one, or both. The type `Test` is given to you: it is a function from a student to `true` or `false`. So are the two helpers `inDepartment` and `atLeast`, which you can build tests with.

B1 `not(test)`

Write a function that takes a test on a student and returns the opposite test: one that passes exactly where the one you were given fails.

```
const inCs = inDepartment('Comp. Sci.');

students.filter(inCs).length        // 4
students.filter(not(inCs)).length   // 9
```

What you hand back is a function, not a `true` or a `false`.

B2 `partition(list, test)`

Write a function that takes a list of students and a test, and returns two arrays: (1) the students that pass the test, and (2) the students that fail it. Go through the list once only. Return the pair as a tuple, so that the two arrays cannot be swapped by mistake and the caller can take them apart by position.

```
const [upper, lower] = partition(students, atLeast(60));

upper.length, lower.length   // 6 7
```

Calling `filter` twice is two passes over the list, so that is not an answer here. Use instead `reduce`, which will do it in one pass.

B3 `once(calculation)`

The parameter calculation is a function that takes no arguments and returns a number. Write a function that takes such a function and returns a new one. The new function runs the calculation the first time it is called and never again: every later call hands back the number from the first call, without calculating it a second time.

```
let calls = 0;
const count = once(() => { calls++; return students.length; });

count(), count(), calls   // 13 13 1
```

The `calls` counter is not part of your work. It belongs to the test, and it is there to prove the calculation ran only once. The calculation adds one to it every time it runs, so in the result the first two numbers are the answer handed back twice, and the last one is how many times it actually ran. If that last number is 2, you are calculating on every call.

Be ready to say where the saved number lives after `once` has returned.

B4 `allOf(...tests)`

Write a function that takes any number of tests and returns a single test that passes only when all of them pass.

```
const senior = allOf(inDepartment('Comp. Sci.'), atLeast(60));

students.filter(senior).length    // 1
students.filter(allOf()).length   // 13
```

Use the three dots in the parameter list to gather the arguments into an array. With no tests at all, every student passes: be able to say why that is the sensible answer and not an accident.

Handing in

Push your solution to your group's Git repository and hand in the link before the next session (One link per group). Make sure that repository is public.